MODULO 4 · LEZIONE 8

MCP: collegare il coding agent e l'assistente alle app

Agenda

Il codice vostro e le app degli altri	teoria	3–5
Il protocollo e il login	teoria	6–12
Un'app nel coding agent	demo	13–14
La stessa app nell'assistente	codice spiegato	15–18
Le note da verificare nella vostra app	lavoro vostro	19–21
Rientro: costo e permessi	discussione	22–24
Il lab della prossima lezione	teoria	25

Prima del codice dell'assistente preparate la cartella: nella dispensa, «Preparare la cartella» e «Preparare l'ambiente».



IL CODICE VOSTRO E LE APP DEGLI ALTRI

Il conto delle note spese è codice vostro

La lezione scorsa la stessa regola stava in due posti. Per il codice vostro non serve un protocollo nuovo.

Nell'assistente: un function tool

```
@function_tool
def note_fuori_massimale(mese: str) -> str:
    """Le note spese fuori massimale di un mese …"""
    return testo_fuori_massimale(mese) # rimborsi.py
```

Il conto lo fa rimborsi.py, dentro il processo dell'assistente.

Nel coding agent: lo script, con Bash

```
python rimborsi.py
```

Come nella skill report-mensile: il programma lo lancia il coding agent, non voi.

Un function tool o uno script bastano quando il codice è vostro.



IL CODICE VOSTRO E LE APP DEGLI ALTRI

Le app degli altri: qui serve MCP

L'ufficio lavora anche su app di altri. Il codice e l'account sono del fornitore, e il fornitore dà un server MCP.

Trello

le board, le liste e le card dell'ufficio

`mcp.trello.com/v1`

Notion

le pagine e i database

`mcp.notion.com/mcp`

Airtable

le basi, cioè tabelle condivise

`mcp.airtable.com/mcp`

Gmail

la posta

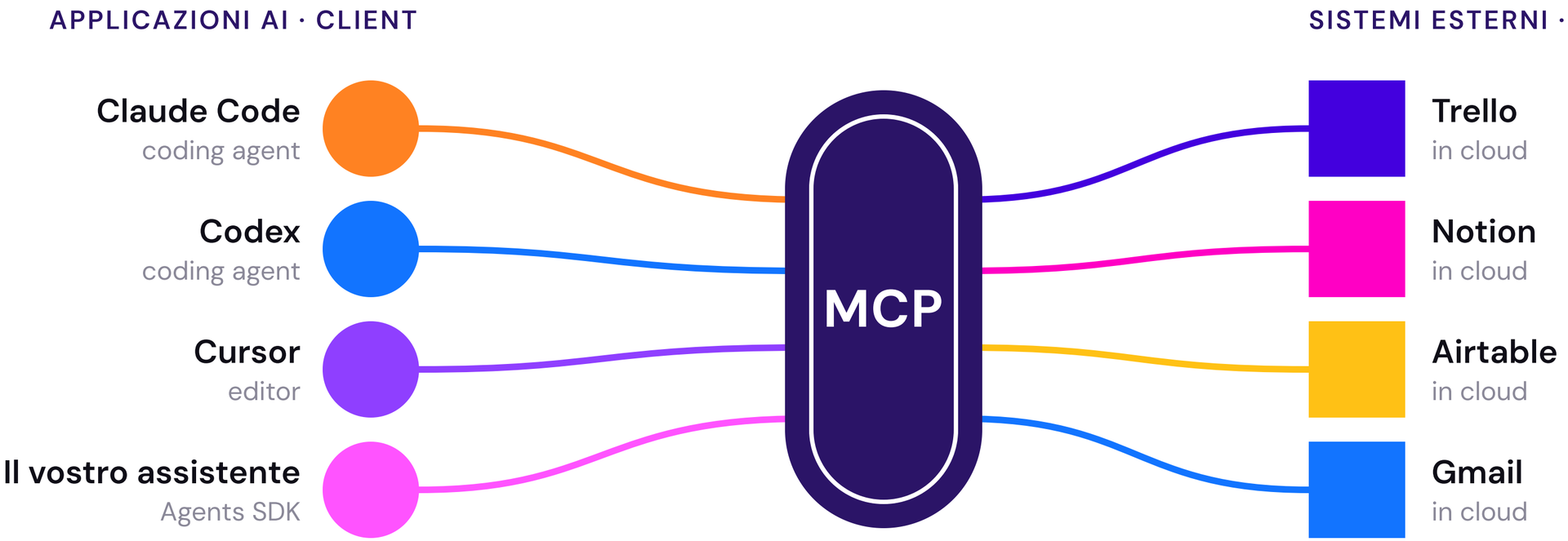
`da claude.ai`

> **Il fornitore scrive il server e lo tiene acceso; voi scrivete solo il suo indirizzo.**



Cos'è MCP

Uno standard aperto per collegare le applicazioni AI a sistemi esterni.



Come una porta USB-C: un attacco solo, per tanti dispositivi.

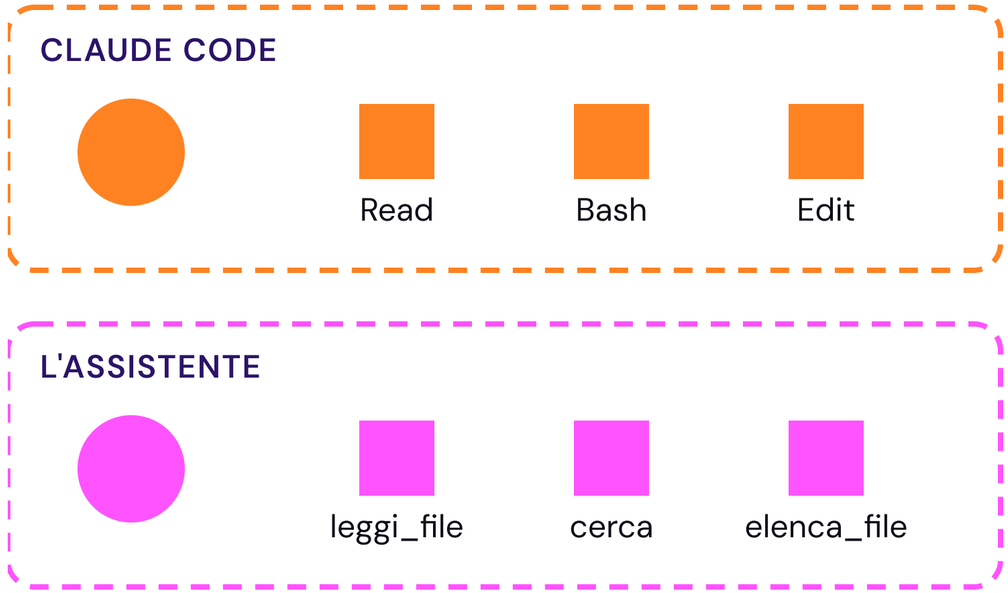
Ogni server offre tool, risorse e prompt. Oggi solo i tool.

IL PROTOCOLLO

Tool del client e tool MCP: dove gira il codice

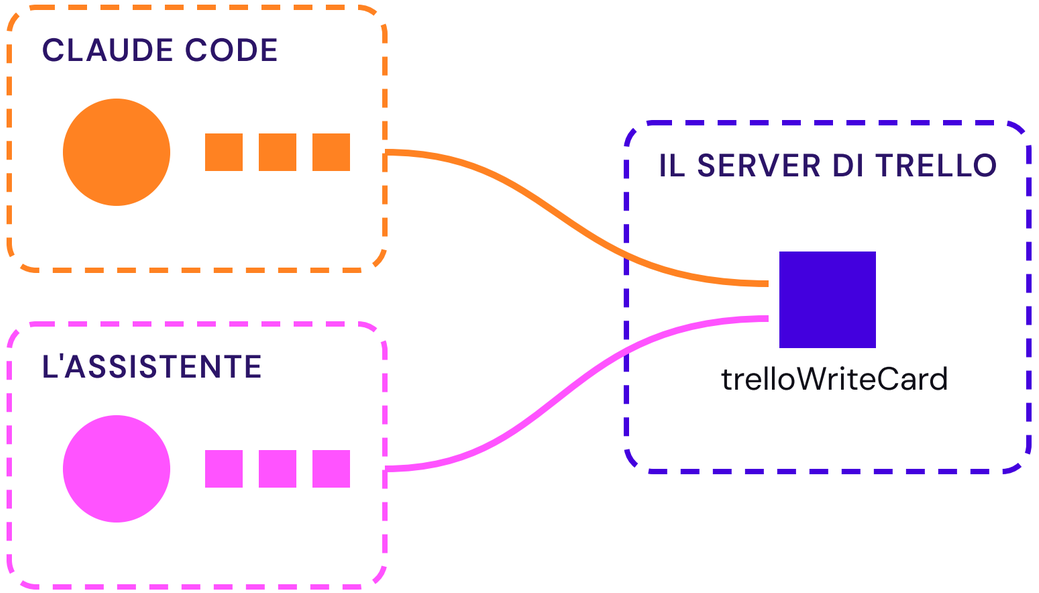
Per il modello sono uguali. Cambia dove gira il codice, e quindi chi lo può usare.

Tool del client



Il codice è dentro il client: lo usa solo lui.

Tool MCP

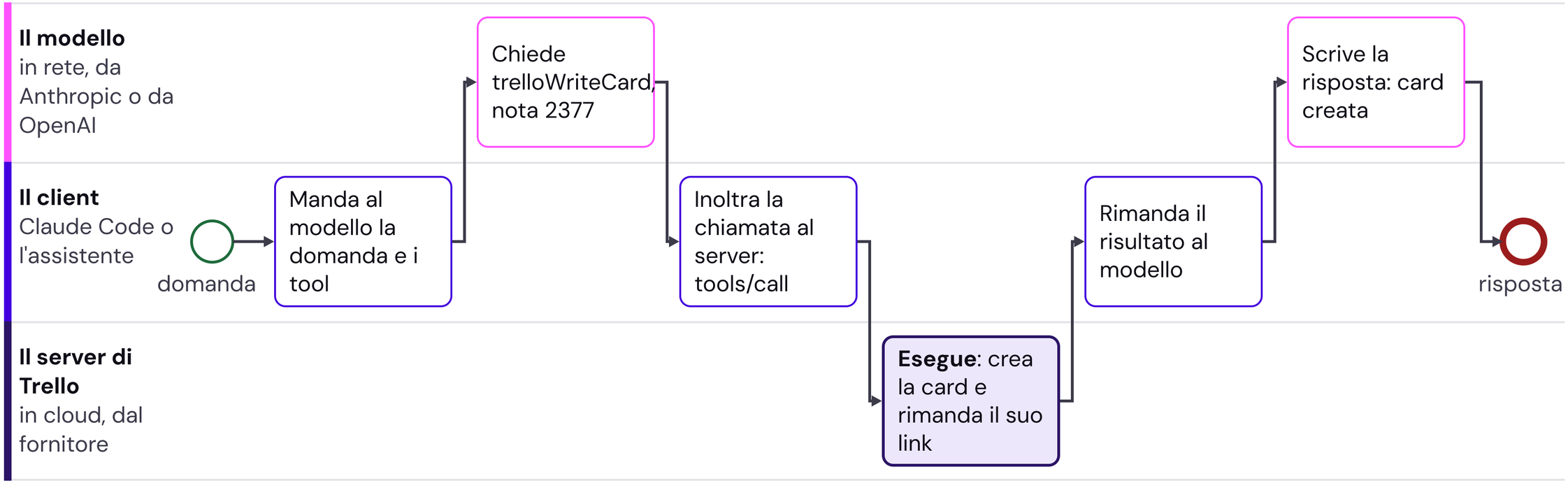


Il codice è nel server, qui del fornitore: lo usa ogni client collegato.

IL PROTOCOLLO

Il modello chiede, il client inoltra, il server esegue

Una domanda, passo per passo: una corsia per attore. Il codice del tool lo esegue solo il server.



Due scenari: il server sul vostro computer o in cloud

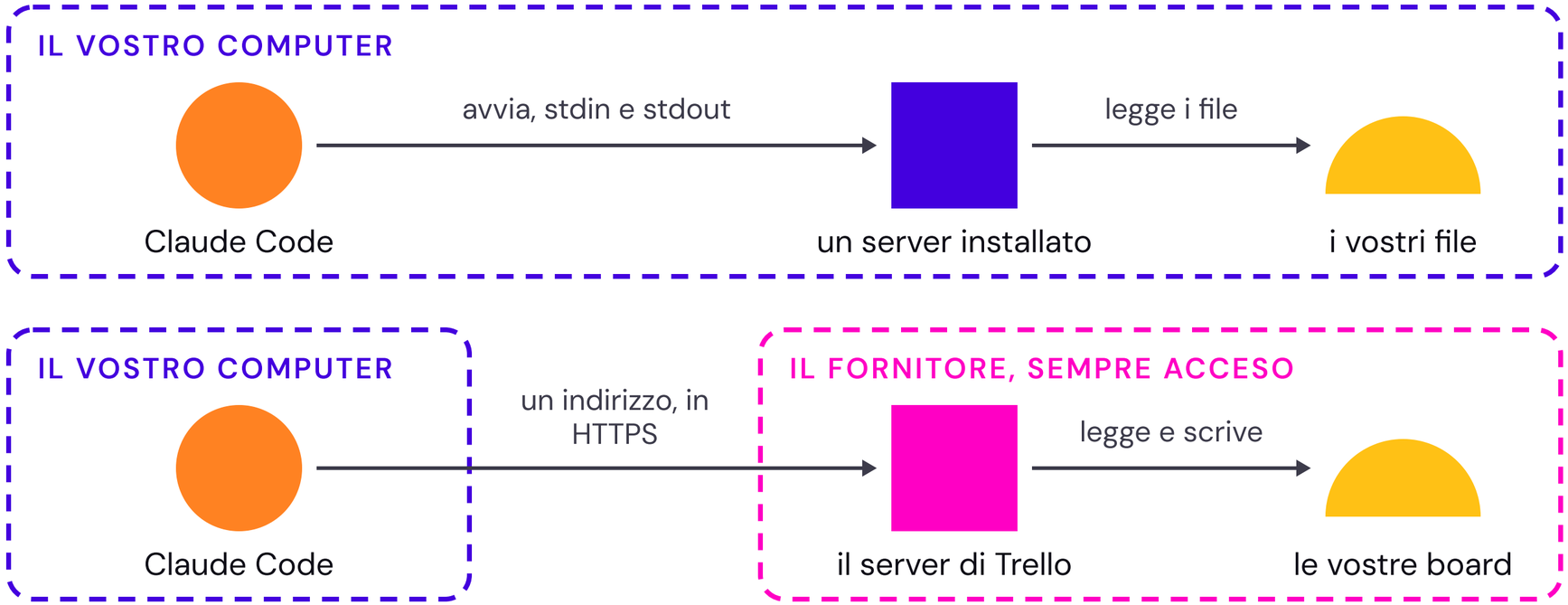
Stesso protocollo, stesse richieste. Cambia dove gira il server.

Locale

un programma sul vostro disco

In cloud

Trello, Notion, Airtable, Gmail



Locale o in cloud: cosa cambia

	Locale	In cloud
Chi lo avvia	il client, quando parte	il fornitore: è sempre acceso
Dove stanno i dati	sul vostro computer, letti con i vostri permessi	dal fornitore; al server arrivano gli argomenti del modello
Come si entra	nessun login: è un vostro programma	con il vostro account, la prima volta dal browser

Il protocollo è lo stesso: tools/list e tools/call.

IL PROTOCOLLO E IL LOGIN

Il login lo fa il client, non il modello

OAuth: il modo standard per dare a un programma l'accesso al vostro account, senza dargli la password.

1

Il server risponde 401

«non autorizzato», e dice dove si fa il login.

2

Il client si registra

presso il fornitore, da solo: registrazione dinamica.

3

Il browser

Voi entrate e acconsentite; un codice di autorizzazione torna al client.

4

Il token

Il client scambia il codice con il token, lo salva e lo manda a ogni richiesta.

```
POST https://mcp.trello.com/v1 → 401
www-authenticate: Bearer resource_metadata="https://mcp.trello.com/.well-known/oauth-protected-resource/v1"
```

Il token è una chiave del vostro account: si tiene fuori da git e non si incolla nella chat.



IL PROTOCOLLO E IL LOGIN

I messaggi veri tra l'assistente e Trello

Registrati dal client: → parte dall'assistente, ← risponde il server di Trello.

```
→ server/discover {}
← 400 «Request must be an initialize request if no session ID is provided.»
→ initialize {"protocolVersion": "2025-11-25", …}
← 200 {"serverInfo": {"name": "atlassian-mcp-server", …}}
→ tools/list {}
← 200 15 tool: trelloReadCard, trelloReadBoard, … trelloWriteCard, …
→ tools/call {"name": "trelloReadBoard", "arguments": {"action": "list", "limit": 100}}
← 200 {"content": [{"type": "text", "text": "{ \"boards\": [ … ] }"}], "isError": false}
```

1. Le prime righe: client e server si accordano sulla versione. Il 400 è previsto.
2. tools/list: l'elenco dei tool, una volta.
3. tools/call: una chiamata per ogni tool chiesto dal modello.



IL PROTOCOLLO E IL LOGIN

Al modello arriva un tool come gli altri

La prima richiesta dell'assistente all'API di OpenAI, accorciata: i tool di Trello accanto ai nostri.

```
"tools": [ … elenca_file, cerca, leggi_file, note_fuori_massimale,
  {"name": "trelloReadBoard",
   "parameters": {"type": "object", "properties": {
      "action": {"type": "string",
                 "enum": ["list", "list_by_workspace", "get", …],
                 "description": "Action to perform. \"list\" — paginated
                    boards the authenticated user is a member of …"},
      …}},
   "strict": false, "type": "function", "description": "Read Trello boards …"}, …]
```

1 Le descrizioni le scrive il fornitore: lunghe, e in inglese.

2 In Claude Code: mcp__claude_ai_Trello__trelloReadBoard, caricato con ToolSearch.

Il modello non parla con il server: chiede il tool, come gli altri.



UN'APP NEL CODING AGENT

Collegare Trello a Claude Code: il connettore di claude.ai

Il login all'app si fa una volta, su claude.ai; Claude Code usa quello.

1

Su claude.ai

Impostazioni → Connettori → Sfoglia → Trello → Connetti: il login sul sito.

2

In Claude Code

Nell'app: il + accanto al campo di testo, poi i connettori. Da terminale c'è già.

3

Nella sessione

Chiedete «quali connettori hai?»: c'è Trello. Da terminale anche /mcp.

```
claude.ai Trello: https://mcp.trello.com/v1 - ✔ Connected
```

Claude Code da terminale, senza connettore: claude mcp add --transport http trello https://mcp.trello.com/v1, poi /mcp per il login.



UN'APP NEL CODING AGENT

Le note fuori massimale in Trello, dal coding agent

La nostra prova, con la richiesta della dispensa e una board di prova: i tool chiamati, in ordine.

```
Read rimborsi.py
Bash ../../../.venv/bin/python rimborsi.py | grep 2026-07 … # il conto
ToolSearch # carica i tool di Trello, differiti
trelloSearch, trelloWriteBoard «Valtesa · prova Claude Code», trelloWriteList «Da verificare»
trelloWriteCard × 20 # una card per nota
```

1. Il conto lo fa rimborsi.py, lanciato con Bash.
2. Le card le crea il server di Trello, a ogni tools/call.
3. In Manual chiede il permesso per ogni tool di Trello, anche per leggere.

Claude Code 2.1.289, Sonnet 5.5, 21/10/2026, in Auto, senza permessi chiesti: 52 secondi, 29 turni.



LA STESSA APP NELL'ASSISTENTE

Il codice: app_agente.py e login_mcp.py

Già scritti, in working/. Li leggiamo insieme: dopo li usate.

> </> **app_agente.py** l'assistente di sempre, più il function tool e il server dell'app. **login_mcp.py** il login OAuth, come lo fa Claude Code.



LA STESSA APP NELL'ASSISTENTE

app_agente.py: il function tool e il server, insieme

Il client MCP dell'SDK per i server in cloud è MCPServerStreamableHttp.

```
APP = {"trello": {"url": "https://mcp.trello.com/v1",
            "tool": ["trelloReadBoard", "trelloWriteCard", …],
            "strutturato": False}, …}

app = APP["trello"]
server = MCPServerStreamableHttp(
    name="trello",
    params={"url": app["url"], "auth": autenticazione("trello", app["url"])},
    tool_filter=create_static_tool_filter(allowed_tool_names=app["tool"]),
    use_structured_content=app["strutturato"])

agente = assistente_base.clone(instructions=… + ISTRUZIONI_APP,
    tools=… + [note_fuori_massimale], mcp_servers=[server])
```

1. auth: il login, da login_mcp.py.
2. tool_filter: solo i tool che servono, su 15.
3. Accanto, il function tool: il codice nostro.



LA STESSA APP NELL'ASSISTENTE

login_mcp.py: il login, prima della sessione

Il client OAuth è nella libreria mcp; login_mcp.py gli dà il browser, la porta e il file del token.

```
def autenticazione(app, url):
    return OAuthClientProvider(server_url=url,
        client_metadata=OAuthClientMetadata(redirect_uris=["http://localhost:8765/callback"], …),
        storage=TokenInFile(file_token(app)), # .mcp-token/trello.json
        redirect_handler=apri_browser, # apre la pagina del login
        callback_handler=aspetta_ritorno) # aspetta il codice su localhost

async def fai_login(app, url):
    if await TokenInFile(file_token(app)).get_tokens():
        return # il token c'è già
    async with httpx2.AsyncClient(auth=autenticazione(app, url), timeout=900) as client:
        await client.post(url, json=richiesta_initialize, …)
```

1. Un login per app, la prima volta: poi usa il token salvato. --login lo rifà.

2. Non è il login del connettore: quello lo tiene claude.ai, e vale solo per Claude Code.

LA STESSA APP NELL'ASSISTENTE

Il tool c'è, ma le istruzioni contano

«Nell'app ci sono già spazi dell'ufficio del personale di Valtesa?», una prova per app.

App	Le istruzioni di sempre	Con ISTRUZIONI_APP
Trello	trova le board di Valtesa	trova le board di Valtesa
Notion	cerca nella cartella dei file, non in Notion	trova la pagina
Airtable	guarda solo i workspace: «non c'è»	trova la base

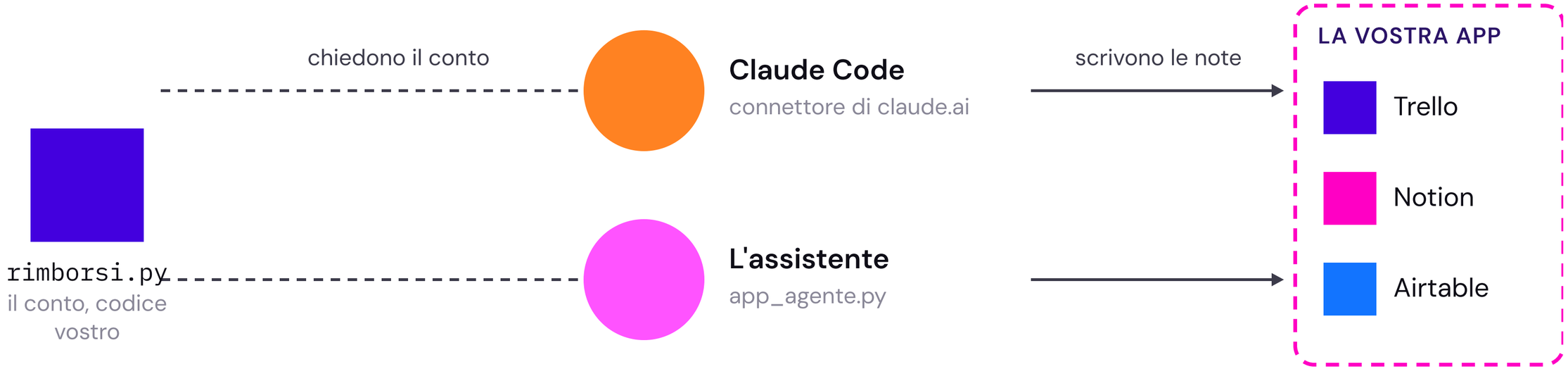
Un tool nuovo va detto anche nelle istruzioni: dove cercare, cosa si può fare.



LE NOTE DA VERIFICARE NELLA VOSTRA APP

Cosa fate: le note di luglio, nell'app che scegliete

Trello, Notion o Airtable. Prima con il coding agent, poi con l'assistente.



Alla fine dite chi ha fatto il conto e chi ha scritto nell'app, nei due client.

LE NOTE DA VERIFICARE NELLA VOSTRA APP

Tre varianti, lo stesso compito

Un account gratuito, il connettore su claude.ai, il login dell'assistente. Se non sapete quale: Trello.

Trello

una board, una lista «Da verificare», una card per nota

```
python app_agente.py trello
```

15 tool; una card per volta. Provato fino in fondo, con tutti e due i client.

Notion

una pagina con un database «Da verificare», una riga per nota

```
python app_agente.py notion
```

65 tool. Nel consenso scegliete le pagine; può creare un database vuoto in più.

Airtable

una base con una tabella «Da verificare», un record per nota

```
python app_agente.py airtable
```

47 tool; risponde solo con dati strutturati: in app_agente.py è già gestito.

Da provare, con lo stesso codice: Todoist, ClickUp, Monday, Jira, Miro, Linear; Asana con MCPServerSse. Indirizzi nella dispensa.



LAVORO VOSTRO

Le note di luglio nella vostra app, da due client

Claude Code e il terminale aperti in working/. Richieste e comandi nella dispensa: «Esercizio».

1

App

Account gratuito e connettore su claude.ai; Claude Code in Manual.

2

Claude Code

La richiesta della dispensa: le note di luglio nell'app.

3

Login

python app_agente.py trello --login (o notion, airtable).

4

Assistente

La stessa richiesta, in uno spazio con un altro nome.

5

Due righe

In esercizio.md: chi ha calcolato, chi ha scritto.

Fatto quando nell'app ci sono le 20 note di luglio due volte, una da Claude Code e una dall'assistente, in due spazi; nella traccia note_fuori_massimale prima del tool dell'app; le due righe in esercizio.md. Poi un commit.

Se avanza tempo: una seconda app, o una non provata dalla dispensa.

Se il login non va: rilanciate --login; se il browser non si apre, copiate l'indirizzo stampato nel terminale.



RIENTRO

Chi ha calcolato e chi ha scritto, nei due client

Le due righe, con Trello. Con Notion e Airtable cambia solo il nome del tool.

Client	Chi ha fatto il conto	Chi ha scritto nell'app
Claude Code	rimborsi.py, lanciato con Bash	trelloWriteCard, eseguito dal server di Trello
L'assistente	note_fuori_massimale: rimborsi.py, nel suo processo	trelloWriteCard, eseguito dal server di Trello

Il conto è codice vostro, le card le scrive il fornitore. Il modello sceglie e chiede.



RIENTRO

Quanto costa un'app nel contesto dell'assistente

L'assistente, gpt-6-luna, con tool_filter. Trello e Airtable mentre scrivono le 20 note; Notion su una domanda di sola lettura.

App	Tool del server	Tenuti	Chiamate al modello	Token in ingresso
Trello	15	6	6	circa 63.000
Airtable	47	6	5	circa 34.000
Notion, in lettura	65	4	4	circa 35.000

L'SDK manda le definizioni dei tool a ogni chiamata: tool_filter le taglia, ma restano lunghe.

Claude Code invece le carica solo quando servono, con ToolSearch.



RIENTRO

I tool che scrivono e le chiavi degli account

Un server dell'app agisce con il vostro account: quello che fa, lo fate voi.

1

In Claude Code

in Manual chiede il permesso per ogni tool dell'app, anche per leggere: sì a leggere e creare, no a cancellare, archiviare e inviare.

2

Nell'assistente

non chiede niente. Il nostro ha solo tool_filter; con require_approval chiederebbe conferma.

3

Il token

è una chiave: .mcp-token/ nel .gitignore, mai nella chat né in un commit.



IL LAB DELLA PROSSIMA LEZIONE

Skill e MCP, nel coding agent e nell'assistente

Quattro ore di lavoro vostro: mettete in pratica le skill e i server MCP.

Con Claude Code

Le skill del progetto e i connettori delle app.

Con l'Agents SDK

Le skill dell'assistente e i server MCP, con il login.



Grazie

La prossima lezione: Lab, skill e MCP nel coding agent e nell'assistente (Modulo 4 · Lezione 9).

Prima: un commit sul branch l08-lavoro, poi git switch main e git pull.